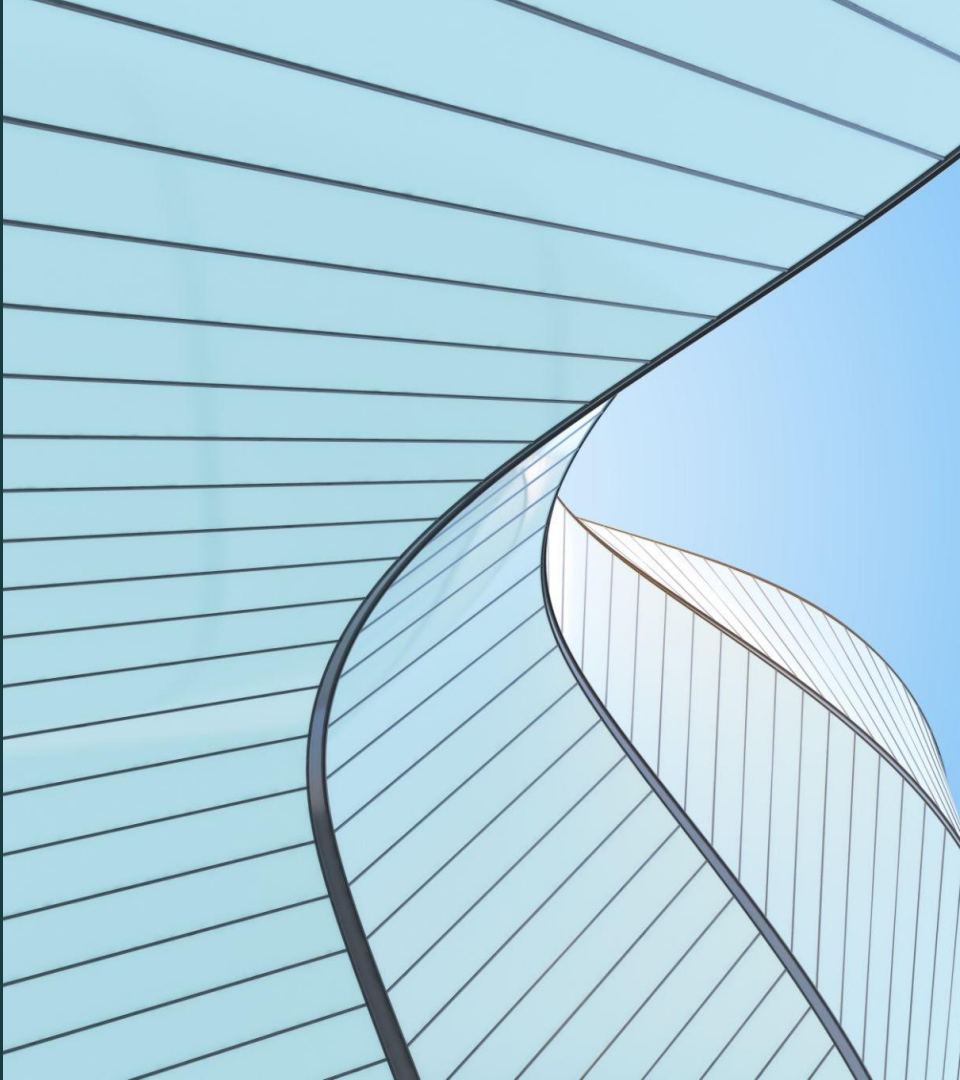


Predicting Successful Albums With Cover Art

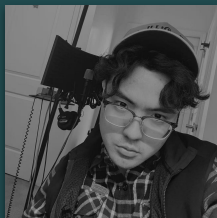
Max Elim, Max Van Zandt, Haoting Yuan, Grant Zhou
December 16, 2025



Our team



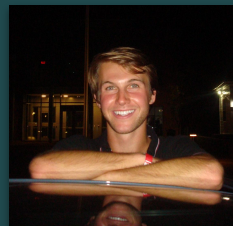
Max Van Zandt



Haoting Yuan



Grant Zhou



Max Elim

Content overview

1

Motivation

2

Data Collection

3

Image Features

4

Clustering

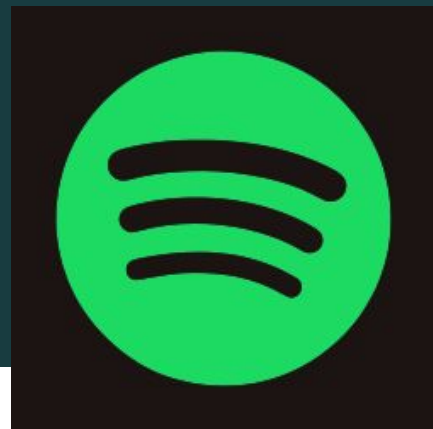
5

Classification

6

Conclusion

Motivation



Music Listeners Are Moving Towards Streaming

Album Cover Art is Prominently Featured on Streaming Platforms

Can we predict album/song success based on cover art/audio features?



Proposed Ideas

Measuring Song Success Based on Album Art

- All songs in an album share the same cover art

Predicting Performance Based On Song Audio Features

- Spotify API deprecated
- Limited data available

Predicting Performance Based on Album Audio Features

- Spotify API deprecated
- Album level features aren't descriptive

Research Question

Can we predict whether an album will enter the Billboard Top 200 based on the visual characteristics of its cover art?



Data Collection



getGenre()
DISCOVER YOUR MUSIC



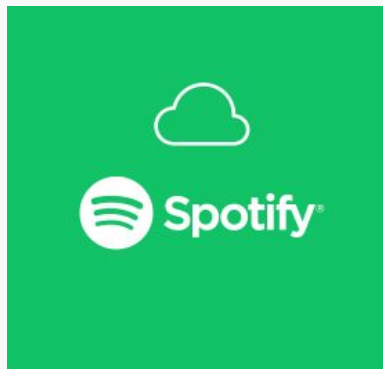
Billboard.py

- Python API for accessing music charts from Billboard.com
- Downloaded all charts from January 2025 – July 2025
 - 461 unique albums
- Extracted album entry days and cover art with Python
- chart_date, rank, title, peakPos, weeks, isNew



Spotify API

- API for user, album, and song data
- Track audio features endpoint deprecated
 - Content can't be used to train AI models
- Album genres endpoint deprecated
- Difficult to query a release date range



genres

array of strings

Required

Deprecated

Deprecated The array is always empty.

Example: `[]`

Get Track's Audio Features

OAuth 2.0

Deprecated

Get audio feature information for a single track identified by its unique Spotify ID.



Important policy note

► Spotify content may not be used to train machine learning or AI model

getgenre.com

- API for AI genre analysis
- ~25% of albums didn't evaluate
- Already very limited on Billboard Albums, couldn't cut



MusicBrainz

- Encyclopedia of music metadata
- Downloaded data of all albums released between December 2024 – July 2025
 - 21522 Albums
- title, artist, date, country, label



Average Time Until Chart

- Albums don't immediately join the chart
- Most enter within about 2–3 weeks
- Subtracted chart date by release date
- Staggered MusicBrainz data collection one month before Billboard

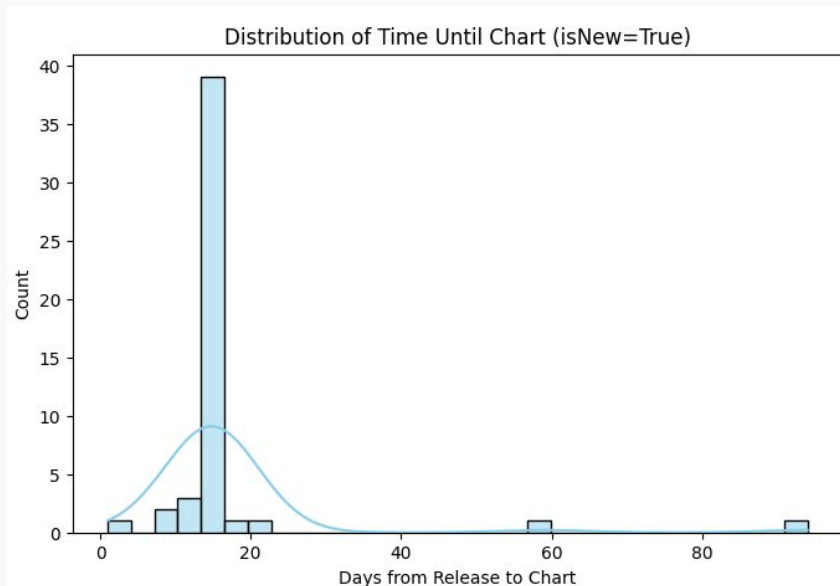
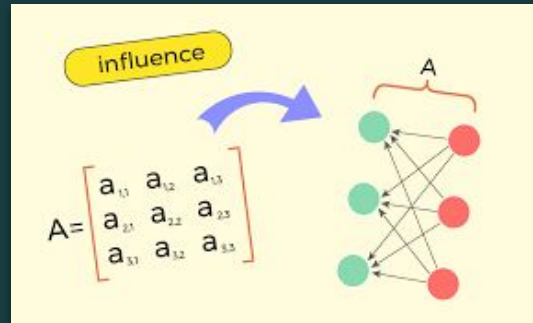


Image Features



RGB Features

$$I_{\text{gray}} = 0.2125R + 0.7154G + 0.0721B$$

- Images are first represented as **RGB**, where each pixel has red, green, and blue intensities
 - RGB features summarize color information using **statistical moments**:
 - Mean → average color intensity
 - Standard deviation → color variability
 - Skewness → asymmetry in color distribution
 - Kurtosis → presence of extreme color values
 - RGB captures *raw color composition*, but not perceptual color qualities



HSV Features

- To capture perceptual color information, images are also transformed into **HSV space**
 - HSV separates color into:
 - **Hue** → dominant color type (e.g., red, blue, yellow)
 - **Saturation** → color intensity vs. grayscale
 - **Value** → brightness
- Each HSV channel is summarized using **histograms**, capturing how pixel values are distributed
 - Color features help distinguish:
 - Bright vs. dark covers
 - Muted vs. vivid designs
 - Warm vs. cool color palette

```
hsv = color.rgb2hsv(img)
hist_h = np.histogram(hsv[...,0], bins=8, range=(0,1), density=True)[0]
hist_s = np.histogram(hsv[...,1], bins=8, range=(0,1), density=True)[0]
hist_v = np.histogram(hsv[...,2], bins=8, range=(0,1), density=True)[0]
```

Edge Features

- Images are converted to **grayscale** so edge detection focuses on structure, not color
- Edge detection is done using **Sobel operators**, which approximate intensity gradients:
 - One kernel measures *horizontal* change
 - One kernel measures *vertical* change
- At each pixel, gradients are combined to compute a **gradient magnitude**
- High gradient magnitude → sharp edges and strong contrast
- Two summary features are extracted:
 - **Average edge strength** across the image
 - **Edge density** → proportion of very strong edges

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * I, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * I$$

Edge Features

- Edge features differentiate:
 - Minimalist covers vs. busy designs
 - Soft imagery vs. sharp typography and outlines



Texture Features

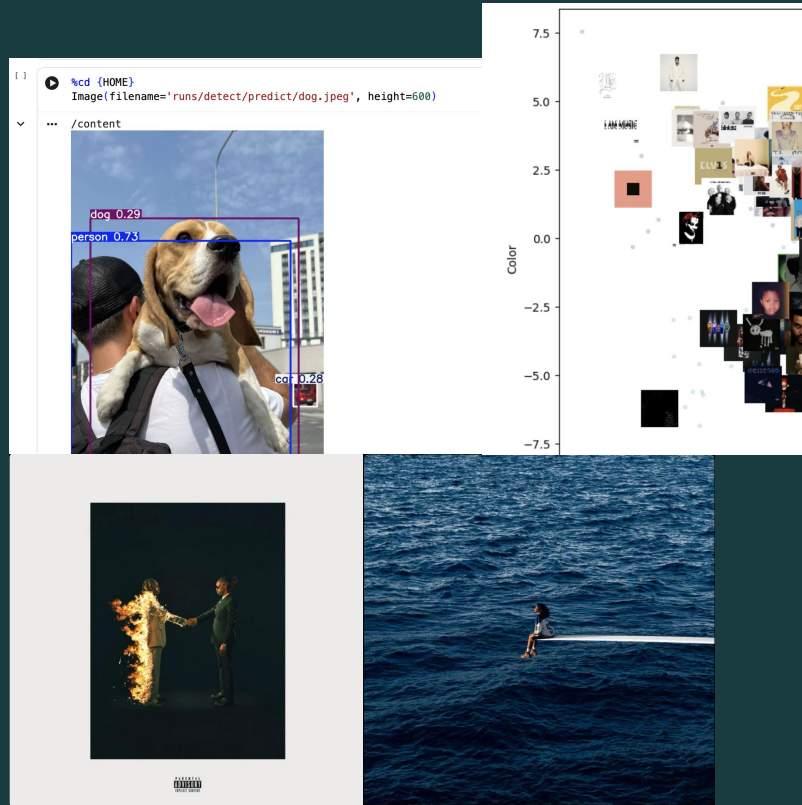
- Texture describes how pixel intensities vary *locally*, not globally
- For each pixel, brightness is compared to its **8 neighboring pixels**
- These comparisons form a **binary pattern** encoding local structure
- Each pattern reflects whether neighboring pixels are brighter or darker than the center
- Repeating this across the image produces a map of local texture codes summarized using a **histogram of pattern frequencies**
 - Lower texture values → smooth, uniform regions
 - Higher texture values → complex patterns and fine detail
- Texture features capture:
 - Graininess
 - Repeated visual motifs
 - Rough vs. smooth visual styles

$$\text{LBP}(x_c, y_c) = \sum_{p=0}^7 s(g_p - g_c) \cdot 2^p$$

where g_c is the center pixel intensity, g_p are the 8 neighbor intensities, and

$$s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Clustering



Data Preprocessing + Feature Selection

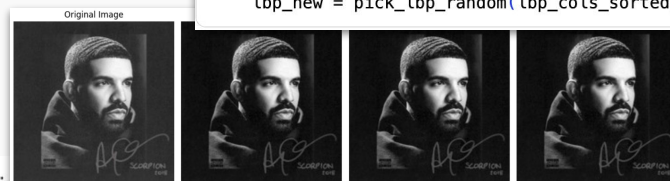
- **Dataset:** 461 Billboard album covers (2025)
- **Visual feature blocks:**
 - Color statistics (RGB mean, skewness)
 - HSV saturation/value bins
 - Texture descriptors (LBP)
- **Preprocessing:**
 - Feature selection
 - LBP *subsampling* Standardization (z-score)

p0	p1	p2
p7	gc	p3
p6	p5	p4

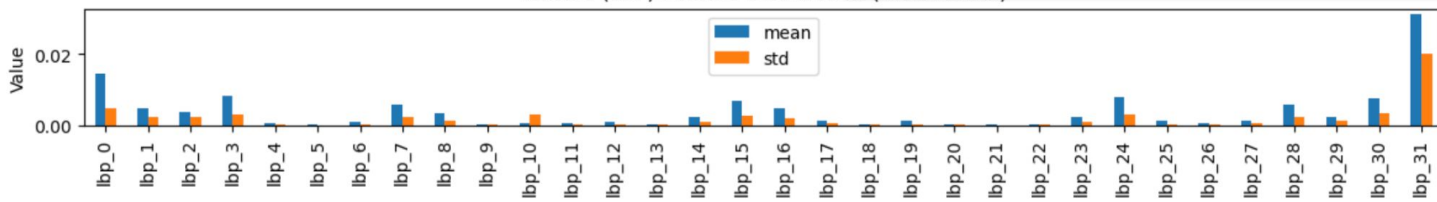
lbp_0=00000000
lbp_31=00011111

```
# sampling method 1
# randomly keep K LBP features
def pick_lbp_random(lbp_list, k, seed=42):
    rng = np.random.default_rng(seed)
    chosen = rng.choice(lbp_list, size=k, replace=False)
    return sorted(chosen, key=lambda x: int(x.split("_")[1]))

lbp_new = pick_lbp_random(lbp_cols_sorted, k=17, seed=0)
```



Texture (LBP) Feature Distribution (mean $\pm$ std)

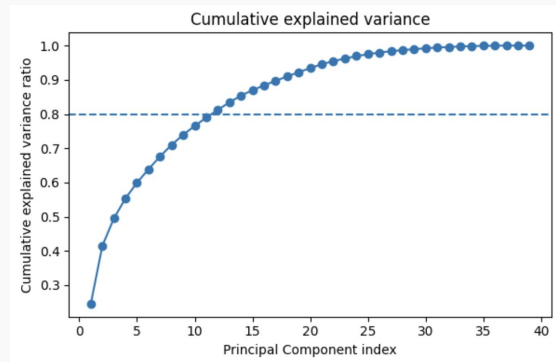
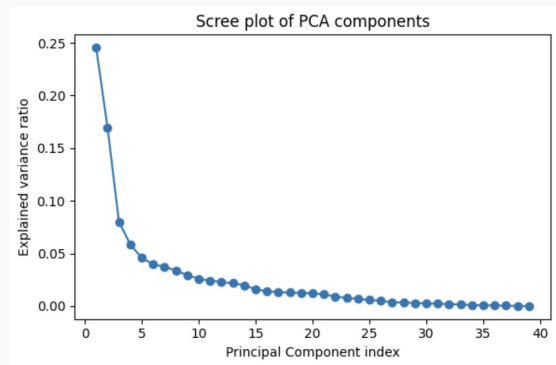


PCA Loadings & Visual Semantics

- **Dimensionality reduction:**
 - PCA on 39 standardized visual features
 - Clustering model: K-means on PCA embeddings
- **Explained variance:**
 - PC1 $\approx$ 24.5%, PC2 $\approx$ 17.0%
 - First 2 PCs explain **$\sim 41.5\%$** of variance
- **PC1:** Texture-dominant axis
 - Driven by LBP features (lbp_0, lbp_4, lbp_6, lbp_13)
- **PC2:** Color-dominant axis
 - Driven by RGB mean & skewness

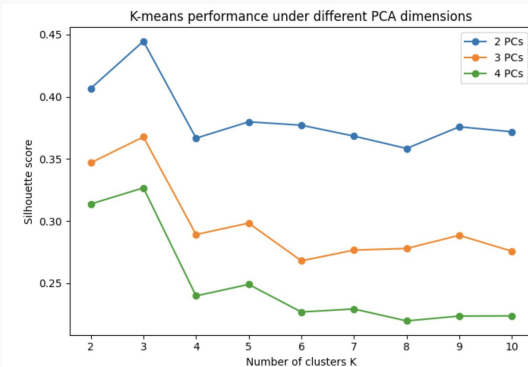
Interpretation:

1. PC1 $\approx$ visual complexity / texture density
2. PC2 $\approx$ color brightness & chromatic balance

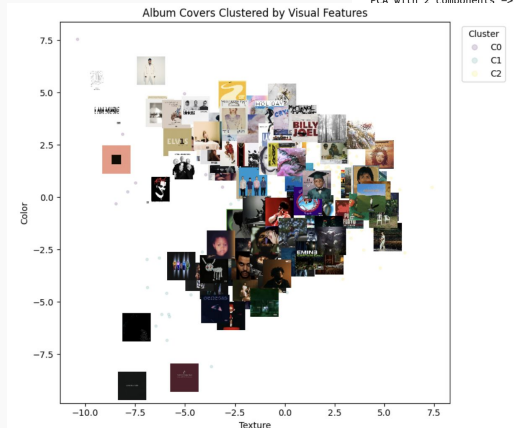
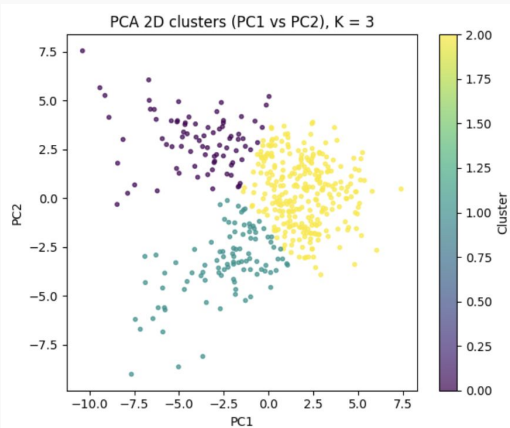


From Clusters to Design & Supervised Learning I

- **Optimal clustering:**
PCA(2) + K-means, K = 3 Silhouette $\approx$ 0.445 (best among tested setups)
- **Cluster characteristics:**
 - Cluster 0: clarity-low-texture,
 - Cluster 1: dark vibe-low texture
 - Cluster 2: colorful, mid2high-texture



PCA with 2 components $\rightarrow$ best K = 3, silhouette = 0.445
best K = 3, silhouette = 0.368
best K = 3, silhouette = 0.327



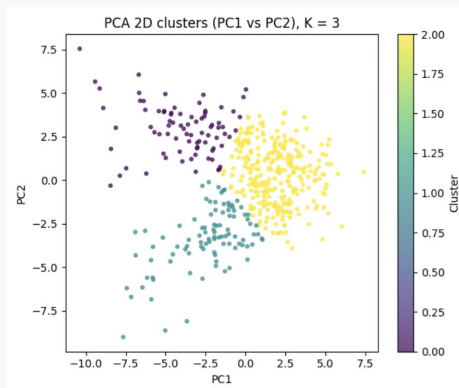
From Clusters to Design & Supervised Learning II

- **Design insights**

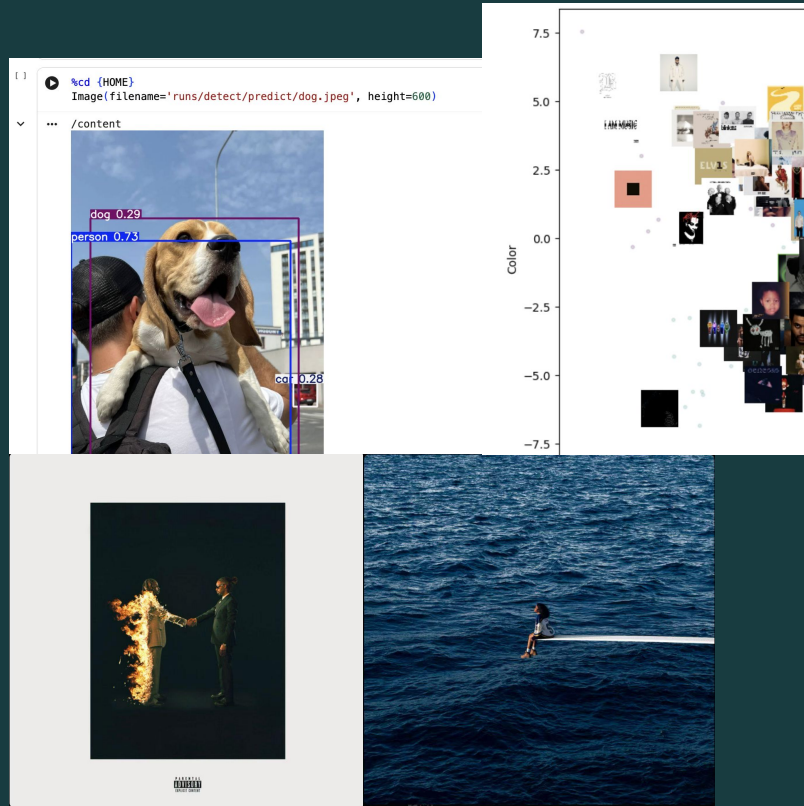
- **White-space regions in PC space:** sparse / empty areas indicate rare visual combinations under current dataset coverage.
- **Nearest-neighbor design retrieval:** a new cover can be projected into the same PCA space and matched to the nearest N covers by Euclidean distance, enabling composition/layout inspiration based on color + texture similarity.

- **Potential link to supervised learning (4 future works)**

- After expanding the dataset, we can analyze: Which clusters dominate among correctly classified albums to test whether certain visual regimes are easier/harder to predict.



Classification



YOLO-Based Semantic Features

1. What is YOLO?

- YOLO (*You Only Look Once*) is a real-time object detection model
- It detects **objects, faces, and spatial layouts** directly from images
- Outputs structured semantic information beyond raw pixels

3. YOLO features used in this project

- `has_person`: whether a human figure is detected
- `face_flag`: presence of a detectable
- `face_person_area_ratio`: proportion of image occupied by people
- `object_diversity`: number of distinct detected object categories
- `subject_cx`: horizontal position of main subject (composition)

2. Why YOLO for album covers?

- Pixel features capture how an image looks
- YOLO features capture what is depicted and where

Classification: Data, Features, Evaluation

- **Goal:** Predict whether a new release is “successful” from cover-art visual features
- **Labels** (Jan 1–Jul 1, 2025):
 - **Success ($y=1$):** Albums appearing on any Billboard 200 chart in the window; deduplicated $\rightarrow$ 461 unique
 - **Normal ($y=0$):** MusicBrainz releases (Dec 1, 2024–Jul 1, 2025), sample 2000, $\rightarrow$ 1623 after data cleaning+removing Billboard overlaps
 - **Final:** 2084 albums, positive rate $\approx$ 22%
- **Features:** 79 hand-crafted cover-art features (RGB stats, HSV, texture LBP bins, edges, YOLO) $\rightarrow X \in \mathbb{R}^{(2084 \times 79)}$
- **Train/test + evaluation:**
 - 80/20 stratified split (train 1667, test 417);
 - report accuracy, precision/recall/F1, confusion matrix, ROC–AUC, ROC curve;

Classification: Models

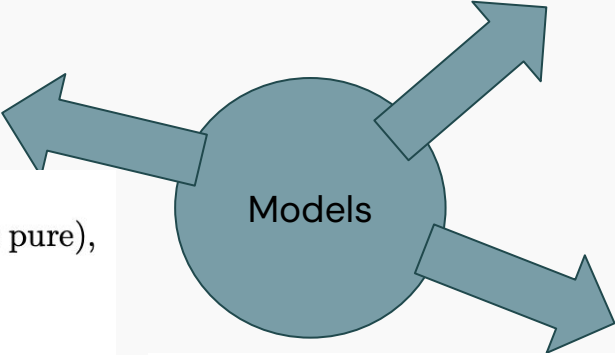
L1-regularized logistic regression

- `penalty = "l1",`
- `solver = "saga",`
- regularization strength `C = 0.5,`
- `class_weight = "balanced"` to compensate for class imbalance,
- `max_iter = 5000.`

Random forest

- `n_estimators = 400,`
- `max_depth = None` (trees grow until leaves are pure),
- `min_samples_leaf = 2` to reduce overfitting,
- `class_weight = "balanced",`
- `n_jobs = -1` to use all CPU cores.

Models



Gradient boosting

- `n_estimators = 250,`
- `learning_rate = 0.05,`
- base tree depth `max_depth = 3,`
- `subsample = 0.9` to introduce stochasticity and regularization

Classification: Predictive performance I

Model	Accuracy	ROC-AUC	Precision (success)	Recall (success)
Logistic Regression (L1)	0.8225	0.87	0.579	0.717
Random Forest	0.8058	0.77	0.649	0.261
Gradient Boosting	0.8297	0.79	0.733	0.359

Confusion
Matrices

$$\begin{pmatrix} 277 & 48 \\ 26 & 66 \end{pmatrix}$$

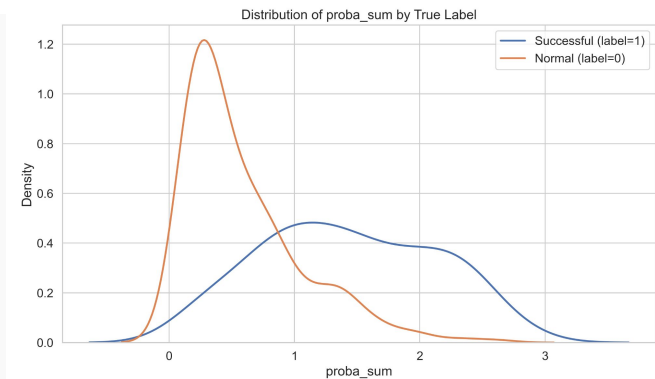
L1-Logit

$$\begin{pmatrix} 312 & 13 \\ 68 & 24 \end{pmatrix}$$

Random
Forest

$$\begin{pmatrix} 313 & 12 \\ 59 & 33 \end{pmatrix}$$

Gradient Boosting



Classification: Predictive performance II

Num non-zero coef: 66 / 79

Top 20 Logistic Regression (L1) feature importances:

feature	coef	coef
lbp_10	1.9046	-1.9046
lbp_27	1.3509	1.3509
lbp_16	1.2901	1.2901
lbp_5	1.2900	1.2900
lbp_4	1.2701	-1.2701
lbp_7	1.1772	1.1772
lbp_1	1.0888	1.0888
lbp_3	1.0516	1.0516

Top 20 Random Forest feature importances:

lbp_23	importance=0.1206
lbp_17	importance=0.0951
lbp_24	importance=0.0682
lbp_28	importance=0.0604
lbp_19	importance=0.0460
lbp_26	importance=0.0303
sobel_mean	importance=0.0302
lbp_11	importance=0.0271
lbp_0	importance=0.0257
lbp_12	importance=0.0196
lbp_29	importance=0.0195
lbp_18	importance=0.0191
lbp_10	importance=0.0179
sobel_mean	importance=0.0173
lbp_23	importance=0.0166
has_person	importance=0.0157
subject_cy	importance=0.0156
lbp_31	importance=0.0154
lbp_7	importance=0.0149
lbp_30	importance=0.0148
lbp_18	importance=0.0144
lbp_1	importance=0.0140

Top 20 Gradient Boosting feature importances:

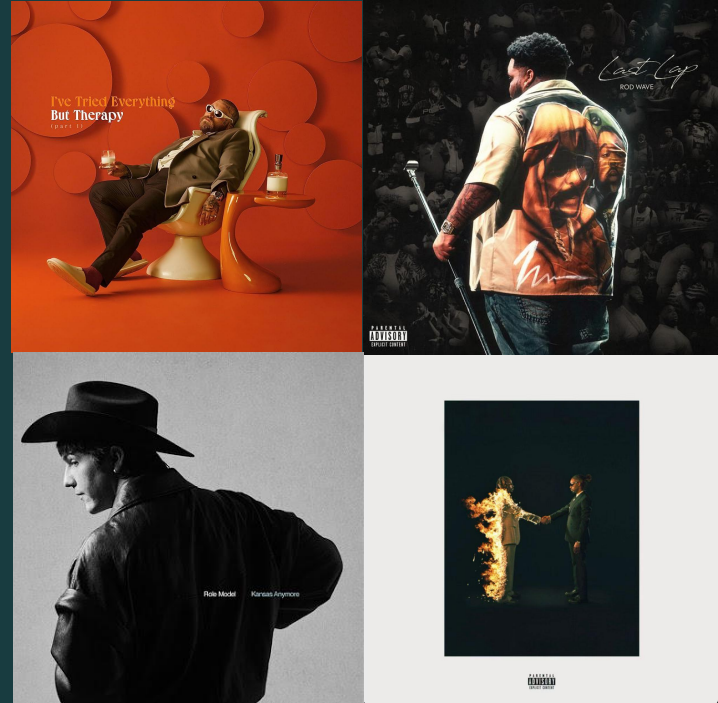
lbp_26	importance=0.1206
person_area_ratio	importance=0.0951
lbp_11	importance=0.0682
lbp_27	importance=0.0604
lbp_7	importance=0.0460
lbp_31	importance=0.0303
lbp_8	importance=0.0302
lbp_14	importance=0.0271
lbp_10	importance=0.0257
lbp_18	importance=0.0196
lbp_25	importance=0.0195
lbp_5	importance=0.0191
sobel_mean	importance=0.0179
subject_cy	importance=0.0173
lbp_16	importance=0.0166
h_6	importance=0.0157
h_0	importance=0.0147
rgb_mean_B	importance=0.0146
lbp_19	importance=0.0146
lbp_3	importance=0.0144

importance=0.0172
importance=0.0172
importance=0.0157
importance=0.0156
importance=0.0154
importance=0.0149
importance=0.0148
importance=0.0144
importance=0.0140

Taken together, the three models **agree** that:

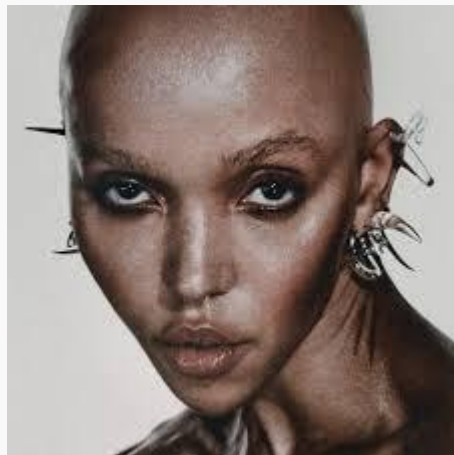
1. **Local texture patterns** (LBP features),
2. **Edge strength and structural complexity**, and
3. The **presence, size**, and placement of **human figures** are the primary visual cues associated with Billboard success in this data.

Conclusion



Takeaway: “What separates a ‘hit-looking’ cover from a ‘non-hit-looking’ one?”

- **Faces and people matter:** covers with large, centered faces are much more likely to be classified as hits.
- **Composition matters:** strong focal points near the center or rule-of-thirds lines correlate with success; diffuse or ambiguous compositions correlate with non-hits.
- **Texture and edges matter:** clean, well-defined textures and edges appear in hits; noisy or low-contrast textures appear in non-hits.
- **Color choices matter:** bold, saturated, or high-contrast palettes look “chart-like,” while muted or experimental palettes look niche.
- Across both groups, the models are not using a single feature, but a **consistent combination of texture, edges, color, and human presence** to decide whether a cover “looks like” a Billboard album.



FKA Twigs, Eusexua

Limitations

- Relatively narrow definition of success
 - Must reach the Billboard Top 200 between January and July of 2025
- Assuming cover art contains signals relating to success
 - What about the effect of artist popularity & social media?
- Hand-crafted visual features
 - Do they capture the visual characteristics sufficiently?



Thank you!